

BAB VIII

LANJUTAN ANALISIS ASOSIASI

A. Capaian Pembelajaran

Pada pertemuan ini mahasiswa/i diharapkan dapat menganalisa:

1. Frequent ItemSet Generation
2. Rule of Generation

B. Materi

1. Frequent ItemSet Generation

Untuk menghasilkan suatu itemset maka diperlukan suatu teknik atau metode yaitu frequent itemset mining. Pola penambangan objek sering mengarah pada penemuan asosiasi dan korelasi antara itemset dalam kumpulan besar kumpulan data transaksional atau relasional, dan sejumlah besar data dikumpulkan dan disimpan secara terus menerus. Banyak industri tertarik untuk mengekstrak pola tersebut dari database mereka. (Han et al., 2011)

Tujuannya dari pola penambangan objek, yang sering mengarah pada penemuan asosiasi dan korelasi antara objek dalam peristiwa besar atau kumpulan data relasional. Dengan sejumlah besar data yang terus dikumpulkan dan disimpan, banyak industri tertarik untuk mengekstraksi pola semacam itu dari database mereka. Menemukan korelasi yang menarik antara beberapa catatan transaksi bisnis dapat mendukung banyak proses pengambilan keputusan bisnis perusahaan, seperti perencanaan inventaris, pemasaran silang, dan analisis perilaku pembelian pelanggan. Terdapat 3 kategori pada pola penambangan itemset :

- a. Algoritma mining Frequent Itemset (FI)
- b. Algoritma mining Frequent Closed Itemset (FCI)
- c. Algoritma Maximal Frequent Itemset (MFI).

Dengan aturan Frequent Closed Itemset (FCI) adalah subset dari Frequent Itemset (FI) dan Maximal Frequent Itemset (MFI) merupakan subset dari Frequent Itemset (FI) dan Frequent Closed Itemset (FCI), dengan symbol ($MFI \subseteq FCI \subseteq FI$). Contoh $A=\{1, 2, 3,...x\}$ merupakan item-item yang terdapat dalam sebuah database dan $B=\{1, 2, 3,...y\}$ merupakan kumpulan nomer dari sebuah transaksi (Tset). Maka disimpulkan bahwa itemset X dapat didefinisikan sebagai $X \subseteq A$ dan sebuah transaksi (Tset) Y dapat didefinisikan sebagai $Y \subseteq B$. Item X didefinisikan sebagai $\sigma(X)$ jika terdapat jumlah transaksi di dalamnya terdapat support item X.

Jika support X sama dengan nilai minimum support atau lebih besar maka sebuah itemset X dapat disebut sebagai frequent itemset, maka bisa dispesifikasikan menjadi ($\sigma(X) \geq \text{min_support}$). Untuk menentukan itemset tersebut frequent atau tidak dapat dilihat pada minimum support suatu nilai, yang mana si pengguna dapat menentukan nilai minimum support tersebut. Contoh minimum support bernilai 50% dan terdapat empat transaksi pada database, untuk mengetahui suatu itemset dapat dikatakan frequent, maka di dalam itemset tersebut didapati dua transaksi ($50\% \times 4 = 2$). Mengapa menggunakan algoritma apriori? Karena algoritma apriori merupakan algoritma dasar untuk menemukan frequent itemset. Teknik algoritma apriori sendiri menggunakan pendekatan iteratif yang di kenal sebagai teknik pola pencarian sederhana, di mana k -itemsets digunakan untuk penemuan $(k + 1)$ -itemsets, kemudian $(k+1)$ -kandidat yang ditetapkan dengan frekuensi kejadian kurang atau kurang dari ambang batas (min_support) dipotong dan tidak digunakan untuk menentukan aturan penggabungan.

2. Langkah-langkah menemukan frequent itemset

Dalam menemukan frequent itemset dengan menggunakan algoritma apriori terdapat beberapa Langkah:

- a. Langkah awal adalah mengumpulkan 1 item yang ditemukan dengan memindai database dan mencatat jumlah dan penampilan setiap kategori produk di setiap transaksi.
- b. Langkah kedua, himpunan 1 elemen, hanya untuk menemukan himpunan 2 elemen. Kandidat 2 item yang mungkin ditemukan dengan menghubungkan satu objek ke objek lainnya sehingga diperoleh kemungkinan dua objek yang cocok.
- c. Langkah ketiga yaitu 2-elementary menghitung nilai kemunculannya pada setiap event. Ambang batas (min_support) disetel untuk mengecualikan kandidat potensial yang dianggap langka atau langka.
- d. Pada langkah keempat, himpunan 2 elemen yang memenuhi threshold dihitung menurut nilai support dan confidence-nya. Kumpulan dua elemen yang memenuhi persyaratan dukungan minimum dan min_certainty digunakan sebagai aturan bergabung.
- e. Dan langkah terakhir adalah himpunan 2 elemen, yang digunakan untuk mencari himpunan 3 elemen, dan seterusnya hingga tidak ditemukan lagi $(k+1)$ item.

Untuk meningkatkan efisiensi dalam membangun sekumpulan item berulang, properti penting yang disebut properti apriori digunakan untuk mengurangi ruang pencarian. "Apa yang dimaksud dengan properti

apriori?" Properti apriori adalah semua himpunan bagian yang tidak kosong (ada) dari sekumpulan elemen berulang. Fitur apriori didasarkan pada pengamatan berikut. Menurut definisi, jika sebuah itemset I tidak memenuhi ambang dukungan minimum, $\min \text{sup}$, maka I tidak sering, yaitu, $P(I) < \min \text{support}$. Dimana jika item A ditambahkan ke itemset I , maka itemset yang dihasilkan (yaitu, $I \cup A$) tidak dapat muncul lebih sering daripada I . Oleh karena itu, $I \cup A$ juga tidak sering, yaitu, $P(I \cup A) < \min \text{sup}$. Properti ini milik kategori khusus dari properti yang disebut antimonotonisitas dalam arti bahwa jika suatu himpunan tidak dapat lulus ujian, semua supersetnya juga akan gagal dalam ujian yang sama. "Bagaimana properti Apriori digunakan dalam algoritme?" Untuk memahami ini, mari kita lihat bagaimana L_{k-1} digunakan untuk mencari L_k untuk $k \geq 2$. Proses dua langkah diikuti, terdiri dari join dan prune.

a. **Join Step**, Langkah bergabung: Untuk mencari L_k , sekumpulan calon objek dibangkitkan dengan menggabungkan L_{k-1} dengan dirinya sendiri. Himpunan kandidat ini dilambangkan dengan C_k . Biarkan I_1 dan I_2 menjadi itemset di L_{k-1} . Notasi $li[j]$ merujuk pada item ke- j dalam li (misalnya, $I_1[k-2]$ mengacu pada item kedua hingga terakhir dalam I_1). Untuk implementasi yang efisien, Apriori mengasumsikan bahwa item dalam sebuah transaksi atau itemset disortir dalam urutan leksikografis. Untuk $(k-1)$ -itemset, li , ini berarti bahwa item diurutkan

sedemikian rupa sehingga $li[1] < li[2] < \dots < li[k - 1]$. Penggabungan, $L_{k-1} * L_{k-1}$, dilakukan, di mana anggota L_{k-1} dapat digabungkan jika item pertama ($k - 2$) mereka sama. Artinya, anggota I_1 dan I_2 dari L_{k-1} digabungkan jika $(I_1[1] = I_2[1]) \wedge (I_1[2] = I_2[2]) \wedge \dots \wedge (I_1[k - 2] = I_2[k - 2]) \wedge (I_1[k - 1] < I_2[k - 1])$. Kondisi $I_1[k - 1] < I_2[k - 1]$ hanya memastikan bahwa tidak ada duplikat yang dihasilkan. Itemset hasil yang dibentuk dengan menggabungkan I_1 dan I_2 adalah $\{I_1[1], I_1[2], \dots, I_1[k - 2], I_1[k - 1], I_2[k - 1]\}$.

- b. **Prune Step**, C_k adalah superset dari L_k , anggota-anggotanya dapat diulang atau tidak, tetapi semua k himpunan elementer yang berulang dimuat dalam C_k . Mencari di database untuk menentukan jumlah setiap kandidat dalam C_k mengarah pada penentuan L_k (yaitu semua kandidat yang jumlahnya setidaknya dukungan minimum menurut definisi sering dan karena itu termasuk dalam L_k). Namun, C_k bisa sangat besar, sehingga diperlukan perhitungan yang ekstensif. Untuk memperkecil ukuran C_k , digunakan properti Apriori seperti berikut : Setiap $(k - 1)$ -itemset yang tidak sefrekuensi maka tidak dapat dijadikan subset dari yang frequent k -itemset. Maka itu, seandainya ada $(k - 1)$ -subset dari kandidat k -itemset yang tidak ada di L_{k-1} , maka kandidat tersebut juga tidak bisa frequent sehingga dapat dihapus dari C_k . Pengujian subset ini dapat dilakukan dengan cepat

dengan memelihara pohon hash dari semua frequent itemset.

Banyak variasi dari algoritma Apriori telah diusulkan yang berfokus pada peningkatan efisiensi dari algoritma aslinya. Seperti:

- a. Teknik berbasis hash (hashing itemset ke dalam bucket yang sesuai): Teknik berbasis hash dapat digunakan untuk mengurangi ukuran kandidat k -itemset, C_k , untuk $k > 1$. Misalnya, saat memindai setiap transaksi dalam database untuk menghasilkan sering 1-itemsets, L_1 , kita dapat menghasilkan semua 2-itemsets untuk setiap transaksi, hash (yaitu, memetakan) mereka ke dalam ember yang berbeda dari struktur tabel hash, dan meningkatkan jumlah ember yang sesuai (Tabel 8.1) Sebuah 2-itemset dengan jumlah keranjang yang sesuai dalam tabel hash yang berada di bawah ambang batas dukungan tidak dapat sering terjadi dan karenanya harus dihapus dari kumpulan kandidat. Teknik berbasis hash seperti itu secara substansial dapat mengurangi jumlah calon k -itemset yang diperiksa (terutama ketika $k = 2$).

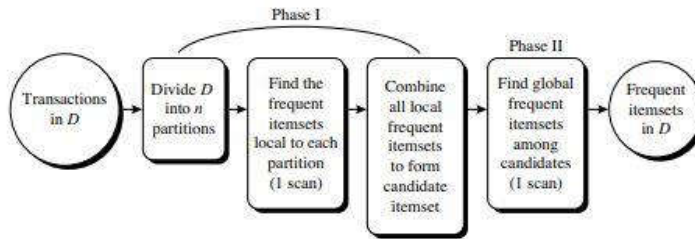
Tabel 8. 1 Teknik berbasis hash

Alamat bucket	0	1	2	3	4	5	6
Urutan bucket	2	2	4	2	2	4	4
Isi bucket	{11,14} {13,15}	{11,15} {11,15}	{12,13} {12,13} {12,13} {12,13}	{12,14} {12,14}	{12,15} {12,15}	{11,12} {11,12} {11,12}	{11,13} {11,13} {11,13}

Pengurangan transaksi (mengurangi jumlah transaksi yang dipindai di iterasi mendatang): Transaksi yang tidak mengandung frequent k-itemsets tidak dapat berisi frequent $(k + 1)$ -itemsets. Oleh karena itu, transaksi semacam itu dapat ditandai atau dihapus dari pertimbangan lebih lanjut karena database selanjutnya memindai j-itemsets, di mana $j > k$, tidak perlu mempertimbangkan transaksi semacam itu.

Mempartisi (mempartisi data untuk menemukan kandidat itemset): Sebuah teknik partisi dapat digunakan yang hanya membutuhkan dua scan database untuk menambang frequent itemsets (Gambar 8.1). Ini terdiri dari dua tahap. Pada fase I, algoritma membagi transaksi D menjadi n partisi yang tidak tumpang tindih. Jika ambang dukungan relatif minimum untuk transaksi di D adalah min sup, maka

jumlah dukungan minimum untuk sebuah partisi adalah $\min \sup \times$ jumlah transaksi di partisi tersebut. Untuk setiap partisi, semua frequent itemset lokal (yaitu, itemset frequent di dalam partisi) ditemukan. Sebuah frequent itemset lokal mungkin atau mungkin tidak sering berkaitan dengan seluruh database, D. Namun, setiap itemset yang berpotensi sering berkaitan dengan D harus terjadi sebagai sering itemset di setidaknya salah satu partisi. 8 Oleh karena itu, semua frequent itemset lokal adalah kandidat itemset sehubungan dengan D. Kumpulan frequent itemset dari semua partisi membentuk kandidat global itemset sehubungan dengan D. Pada fase II, pemindaian kedua D dilakukan di mana dukungan aktual dari setiap kandidat dinilai untuk menentukan frequent itemset global. Ukuran partisi dan jumlah partisi diatur sedemikian rupa sehingga setiap partisi dapat masuk ke dalam memori utama dan karenanya hanya dapat dibaca sekali dalam setiap fase.



Gambar 8. 1 Frequent Itemsets

Sampling (menambang pada subset dari data yang diberikan): Ide dasar dari pendekatan sampling adalah untuk memilih sampel acak S dari data yang diberikan D , dan kemudian mencari frequent itemset di S , bukan D . Dengan cara ini, kita berdagang off beberapa tingkat akurasi terhadap efisiensi. Ukuran sampel S sedemikian rupa sehingga pencarian frequent itemset di S dapat dilakukan di memori utama, sehingga hanya diperlukan satu pemindaian transaksi di S keseluruhan. Karena kita mencari frequent itemset di S daripada di D , ada kemungkinan kita akan kehilangan beberapa frequent itemset global.

Untuk mengurangi kemungkinan ini, kami menggunakan ambang dukungan yang lebih rendah daripada dukungan minimum untuk menemukan kumpulan item yang sering lokal ke S (dilambangkan L^S). Sisa dari database kemudian digunakan untuk

menghitung frekuensi aktual dari setiap itemset di L^s . Mekanisme digunakan untuk menentukan apakah semua frequent itemset global disertakan dalam L^s . Jika L^s benar-benar berisi semua frequent itemset di D , maka hanya diperlukan satu pemindaian D . Jika tidak, pass kedua dapat dilakukan untuk menemukan frequent itemset yang terlewatkan pada pass pertama. Pendekatan pengambilan sampel sangat bermanfaat ketika efisiensi sangat penting seperti dalam aplikasi intensif komputasi yang harus sering dijalankan.

Penghitungan kumpulan item dinamis (menambahkan kumpulan item kandidat pada titik yang berbeda selama pemindaian): Teknik penghitungan kumpulan item dinamis diusulkan di mana database dipartisi menjadi blok-blok yang ditandai dengan titik awal. Dalam variasi ini, kandidat itemset baru dapat ditambahkan pada setiap titik awal, tidak seperti di Apriori, yang menentukan kandidat itemset baru hanya sesaat sebelum setiap pemindaian database lengkap. Teknik ini menggunakan count-so-far sebagai batas bawah dari count yang sebenarnya. Jika count-so-far melewati minimum support, itemset ditambahkan ke dalam frequent itemset collection dan dapat digunakan untuk menghasilkan kandidat yang lebih panjang. Hal ini menyebabkan pemindaian database lebih sedikit

dibandingkan dengan Apriori untuk menemukan semua frequent itemset

3. Rule Generation, compact representation of frequent itemset

Jika frequent itemset yang sering dari transaksi dalam database telah ditemukan, sangat mudah untuk menghasilkan aturan asosiasi yang kuat darinya (jika *rule of associate* telah memenuhi minimum support dan juga minimum confidence). (Han et al., 2011)

Hal Ini dapat dilakukan dengan menggunakan persamaan Generating association rules, untuk sebuah confidence yang akan dibahas kembali dibawah:

$$confidence(A \Rightarrow B) = P(B|A) \frac{support_count(AUB)}{support_count(A)}$$

Probabilitas bersyarat dinyatakan dalam jumlah dukungan itemset, di mana jumlah dukungan (AUB) adalah jumlah transaksi yang berisi kumpulan item AUB , dan jumlah dukungan (A) adalah sebuah isi kumpulan item A yang berasal dari jumlah transaksi. Berdasarkan persamaan ini, aturan asosiasi dapat dihasilkan sebagai berikut:

- a. Untuk setiap frequent itemset I , dibuat tidak kosong dari I dan himpunan bagian.

b. s dari I adalah setiap bagian himpunan kosong,
keluarkan aturannya " $s \Rightarrow (I - s)$ " if $\frac{\text{support_count}(I)}{\text{support_count}(s)} \geq$
 min_conf , dimana min_conf adalah sebuah batas
minimum

Karena aturan dihasilkan dari frequent itemset, masing-masing secara otomatis memenuhi dukungan minimum. Kumpulan item yang sering dapat disimpan sebelumnya dalam tabel hash bersama dengan hitungannya sehingga dapat diakses dengan cepat. Contoh berikutnya berdasarkan data transaksi pada sebuah toko . Di mana data berisi frequent itemset $X = \{I1, I2, I5\}$. Pertanyaannya apa aturan asosiasi yang dapat dihasilkan dari X ? Himpunan bagian tak kosong dari X sebagai berikut $\{I1, I2\}, \{I1, I5\}, \{I2, I5\}, \{I1\}, \{I2\}$, juga $\{I5\}$. Aturan asosiasi yang dihasilkan adalah seperti yang ditunjukkan di bawah ini, masing-masing tercantum secara confidence :

$\{I1, I2\} \Rightarrow I5$, confidence = $2/4 = 50\%$

$\{I1, I5\} \Rightarrow I2$, confidence = $2/2 = 100\%$

$\{I2, I5\} \Rightarrow I1$, confidence = $2/2 = 100\%$

$I1 \Rightarrow \{I2, I5\}$, confidence = $2/6 = 33\%$

$I2 \Rightarrow \{I1, I5\}$, confidence = $2/7 = 29\%$

$I5 \Rightarrow \{I1, I2\}$, confidence = $2/2 = 100\%$

Jika batas minimum confidence, katakanlah hanya 70%, maka hanya ada aturan kedua, ketiga, dan

terakhir yang dihasilkan, sebab ini adalah satu-satunya yang dihasilkan dengan kuat.

C. Latihan

1. Jelaskan definisi mengenai frequent itemset!
2. Sebutkan 5 langkah menemukan frequent itemset!
3. Sebutkan dan jelaskan step property apriori!
4. Apa yang dimaksud dengan rule generation?
5. Apa yang dimaksud dengan probabilitas bersyarat?

D. Referensi

- Ikhwan, A., & Nofriansyah, D. (n.d.). *Penerapan Data Mining dengan Algoritma Fp-Growth untuk Mendukung Strategi Promosi Pendidikan (Studi Kasus Kampus STMIK Triguna Dharma)*.
- Merkwirth, C., Mauser, H., Schulz-Gasen, T., Roche, O., Stahl, M., & Lengauer, T. (2004). Ensemble methods for classification in cheminformatics. *Journal of Chemical Information and Computer Sciences*, 44(6), 1971–1978. <https://doi.org/10.1021/ci049850e>
- Pola, P., Untuk, A., Dengan, P. B., Perbandingan, M., Apriori, A., Salam, A., Zeniarja, J., Wicaksono, W., & Kharisma, L. (2018). GROWTH (STUDY KASUS DISTRO EPO STORE PEMALANG) ASSOCIATION PATTERN MINING FOR PRODUCT ARRANGEMENT USING COMPARISON BETWEEN APRIORI ALGORITHM AND FP-GROWTH (CASE STUDY AT EPO STORE DISTRIBUTION STORE IN PEMALANG). *Jurnal DINAMIK*, 23(2).
- Pristyanto, Y. (2019). PENERAPAN METODE ENSEMBLE UNTUK MENINGKATKAN KINERJA ALGORITME KLASIFIKASI PADA IMBALANCED DATASET. In *Jurnal TEKNOINFO* (Vol. 13, Issue 1). <https://archive.ics.uci.edu/ml/datasets/User+Knowledge>
- Putra, P., H Pardede, A. M., & Syahputra, S. (2022). ANALISIS METODE K-NEAREST NEIGHBOUR

- (KNN) DALAM KLASIFIKASI DATA IRIS BUNGA.
Jurnal Teknik Informatika Kaputama (JTIK), 6(1).
- Siringoringo, R., & Kelana Jaya, I. (2018). *Ensemble Learning Dengan Metode Smote Bagging Pada Klasifikasi Data Tidak Seimbang* (Vol. 3, Issue 2).
- Yunika Hardiyanti, D., Novianti, H., & Rifai, A. (2018). *PENERAPAN ALGORITMA FP-GROWTH PADA SISTEM INFORMASI PERPUSTAKAAN* (Vol. 3, Issue 1).